

Sprawozdanie techniczne

Repozytorium *validationtest*

Dokument opisuje architekturę, pliki, przepływ danych i sposób działania infrastruktury VPN/proxy na Azure.
Data wygenerowania: 14 czerwca 2026

Spis treści

1. Cel projektu i architektura wysokiego poziomu

2. Życie projektu — od zera do działającej usługi

3. Pliki w katalogu głównym

4. Terraform — infrastruktura Azure

5. Skrypty pomocnicze (scripts/)

6. GitHub Actions — workflowy CI/CD

7. Ansible — playbooki i role

8. Pliki tworzone na maszynie wirtualnej

9. Tryby wdrożenia (single / dual)

10.Sekrety i zmienne GitHub

11.Mapa zależności — co z czego wynika

1. Cel projektu i architektura wysokiego poziomu

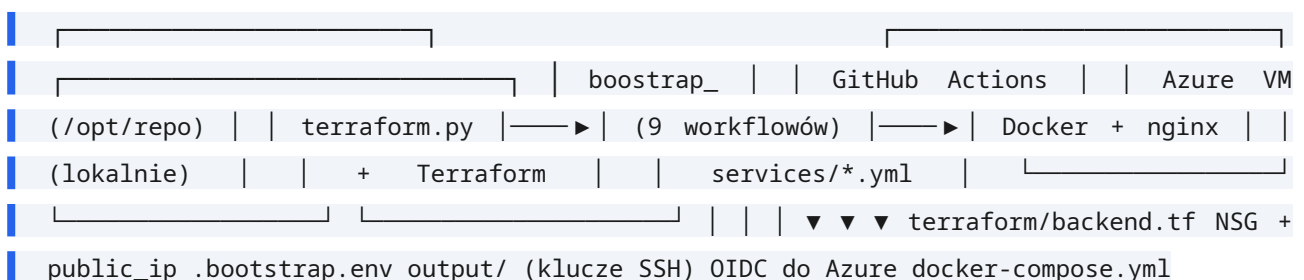
Repozytorium **validationtest** automatyzuje wdrożenie serwera VPN z reverse proxy w chmurze Microsoft Azure. Na jednej maszynie wirtualnej (VM) uruchamiane są kontenery Docker:

- **WG-Easy** — serwer WireGuard (VPN), sieć klientów `10.8.0.0/24`
- **Uptime Kuma** — monitoring dostępności usług
- **nginx** — reverse proxy z certyfikatami TLS (Let's Encrypt)
- **certbot** — pomocniczy kontener do odnawiania certyfikatów

Użytkownik może dodawać własne usługi (np. aplikację na IP klienta VPN `10.8.0.2:8080`), które nginx wystawia na zewnątrz — albo pod własną domeną (HTTPS na porcie 443), albo pod publicznym portem VM (np. `9090`).

Projekt składa się z trzech warstw:

Warstwa	Technologia	Odpowiedzialność
Infrastruktura	Terraform + Azure VM	sieć, firewall (NSG), publiczny IP, FQDN Azure
Konfiguracja VM	Ansible	Docker, nginx, certyfikaty, routing WireGuard, pliki usług
Orkiestracja	GitHub Actions	Bootstrap, deploy, dodawanie/edycja/usuwanie usług, sync NSG



2. Życie projektu — od zera do działającej usługi

Krok 1 — Bootstrap lokalny (jednorazowo)

Na komputerze dewelopera uruchamiasz:

```
python bootstrap.py
```

Skrypt (`bootstrap_terraform.py`) wykonuje:

1. Wykrywa repozytorium GitHub z `git remote`
2. Sprawdza logowanie do Azure (`az login`)
3. Tworzy (lub używa) grupę zasobów `rg-terraform-state` i konto Storage na stan Terraform
4. Generuje `terraform/backend.tf` — konfigurację zdalnego stanu
5. Generuje parę kluczy SSH w `output/id_ed25519`
6. Tworzy aplikację Azure AD `{nazwa-repo}-github-actions` z federacją OIDC dla GitHub Actions
7. Wypisuje wartości do skopiowania jako GitHub Variables i Secrets
8. Opcjonalnie commituje i pushuje `backend.tf`

Krok 2 — Bootstrap VM (workflow GitHub)

Workflow **Bootstrap VM** (`bootstrap-vm.yml`):

1. `terraform apply` — tworzy VM, sieć, NSG
2. Czeka na propagację DNS (do 10 min), jeśli podano własne domeny
3. Łączy się SSH jako `azureuser`
4. Klonuje repo do `/opt/{nazwa-repo}`
5. Zapisuje `.bootstrap.env` (domeny, tryb, podsieci Docker)
6. Uruchamia `ansible-playbook bootstrap.yml`

Krok 3 — Deploy przy pushu

Push na gałąź `main` uruchamia `deploy.yml` (z wyjątkiem samego `backend.tf`). Na VM: `git pull + ansible/deploy.yml`.

Krok 4 — Zarządzanie usługami

Workflowy ręczne: dodaj / edytuj / usuń usługę → plik YAML na VM → synchronizacja NSG → Ansible.

3. Pliki w katalogu głównym

Plik	Opis
<code>bootstrap_terraform.py</code>	Główny skrypt bootstrapu (nazwa z literówką — historycznie). ~700 linii. Działa na Linux i Windows. Tworzy backend Terraform, klucze SSH, aplikację OIDC Azure. Funkcje pomocnicze: <code>build_command()</code> (Windows: <code>cmd /c az.cmd</code>), <code>push_backend_tf()</code> (commit tylko gdy plik się zmienił).
<code>bootstrap.py</code> <code>bootstrap_terraform.py</code>	Cienkie aliasy — wywołują <code>main()</code> z <code>bootstrap_terraform.py</code> .
<code>.env.template</code>	Szablon zmiennych środowiskowych WG-Easy. Przy bootstrapie VM workflow podstawia <code>WG_ADMIN_USER</code> , <code>WG_ADMIN_PASSWORD</code> , <code>WG_DOMAIN</code> , <code>DOCKER_SUBNET</code> przez <code>envsubst</code> i zapisuje jako <code>.env</code> na VM.
<code>.gitignore</code>	Ignoruje <code>output/</code> (klucze SSH, podsumowanie bootstrapu), <code>__pycache__</code> .
<code>README.md</code>	Placeholder — tylko tytuł repozytorium.

Katalog `output/` (lokalny, gitignored)

Plik	Opis
<code>id_ed25519</code>	Klucz prywatny SSH → GitHub Secret <code>SSH_PRIVATE_KEY</code>
<code>id_ed25519.pub</code>	Klucz publiczny → GitHub Variable <code>SSH_PUBLIC_KEY</code>
<code>federation.json</code>	Konfiguracja federacji OIDC Azure AD dla GitHub Actions
<code>bootstrap-summary.txt</code>	Podsumowanie tekstowe: Client ID, Tenant ID, ścieżki plików

4. Terraform — infrastruktura Azure

Katalog `terraform/` definiuje zasoby Azure dla jednej maszyny VPN/proxy.

Pliki Terraform

Plik	Zawartość
provider.tf	Wymaga Terraform ≥ 1.6 , provider azurerm w wersji ~ 4.0 .
backend.tf	Zdalny stan w Azure Storage (generowany przez skrypt bootstrap). Klucz stanu: {nazwa-repo}.tfstate.
variables.tf	Wejścia Terraform (tabela poniżej).
locals.tf	Porty zarezerwowane: 22, 80, 443, 8443, 51820. Filtruje public_ports i nadaje priorytety regułom NSG od 140 w górę.
network.tf	Resource Group, VNet 10.0.0.0/16, subnet 10.0.1.0/24, NSG z regułami: SSH(100), HTTP(110), HTTPS(120), 8443(125, tryb single), WireGuard UDP 51820(130), dynamiczne porty usług(140+).
vm.tf	Publiczny IP (Static, Standard) z opcjonalnym FQDN Azure, NIC, VM Ubuntu 24.04, użytkownik azureuser, logowanie kluczem SSH.
outputs.tf	Wyjścia: public_ip, azure_fqdn, vm_name, azure_subnet, resource_group_name.

Zmienne wejściowe Terraform

Zmienna	Typ	Domyślnie	Znaczenie
resource_group_name	string	—	Nazwa grupy zasobów Azure (wymagana)
location	string	polandcentral	Region Azure
vm_name	string	""	Nazwa maszyny wirtualnej
vm_size	string	Standard_B2ats_v2	Rozmiar VM
ssh_public_key	string	""	Klucz publiczny SSH administratora
single_domain_mode	bool	false	true = otwiera port 8443 w NSG (Kuma na tej samej domenie co WG)

public_ports list(number) [] Dodatkowe porty TCP dla usług bez własnej domeny

Reguły NSG — podsumowanie

Priorytet	Port	Protokół	Kiedy aktywna
100	22	TCP	Zawsze (SSH)
110	80	TCP	Zawsze (HTTP / ACME challenge)
120	443	TCP	Zawsze (HTTPS)
125	8443	TCP	Tylko single_domain_mode = true
130	51820	UDP	Zawsze (WireGuard)
140+	np. 9090	TCP	Dynamicznie z listy public_ports

5. Skrypty pomocnicze (scripts/)

Skrypt	Funkcja
<code>find-bootstrap-env.sh</code>	Szuka pierwszego pliku <code>/opt/*/bootstrap.env</code> na VM. Używany, gdy nazwa katalogu repo może się różnić.
<code>collect-public-ports.sh</code>	Skanuje <code>services/*.yaml</code> na VM. Dla wpisów z <code>has_domain: false</code> zbiera unikalne <code>public_port</code> . Zwraca JSON, np. <code>[9090,8080]</code> . Uruchamiany przez SSH lub lokalnie.
<code>terraform-sync-nsg.sh</code>	Kluczowy skrypt synchronizacji. Łączy się SSH z VM, zbiera porty (<code>collect-public-ports.sh</code>), czyta <code>DEPLOYMENT_MODE</code> z <code>bootstrap.env</code> , uruchamia <code>terraform apply</code> z aktualnymi <code>public_ports</code> i <code>single_domain_mode</code> . Wywoływany przy <code>add/update/delete</code> usługi i zmianie domen.

6. GitHub Actions — workflowy CI/CD

Wszystkie workflowy (poza `destroy`) używają:

- logowania Azure przez OIDC (`ARM_USE_OIDC`, `azure/login@v2`)
- odczytu IP VM z terraform output `-raw public_ip`
- połączenia SSH jako `azureuser` z sekretu `SSH_PRIVATE_KEY`

6.1 bootstrap-vm.yml — Bootstrap VM

Wyzwalacz: ręczny (`workflow_dispatch`)

Wejścia: `resource_group_name`, `vm_name`, `wg_domain` (opcjonalnie), `kuma_domain` (opcjonalnie), `docker_subnet`

Kroki: Terraform apply → walidacja sekretów i podsieci → DNS wait loop → SSH → instalacja git/ansible → klon repo → `bootstrap.env` → `htpasswd` → inventory Ansible → `bootstrap.yml` → znacznik `bootstrap-completed`

Tryb: brak `kuma_domain` → `single` + `single_domain_mode=true`; z `kuma_domain` → `dual`

6.2 deploy.yml — Deploy

Wyzwalacz: push na main (ignoruje zmiany tylko w terraform/backend.tf)

Kroki: checkout → Azure login → terraform init → SSH → git pull → ansible/deploy.yml

6.3 destroy.yml — Destroy Environment

Wyzwalacz: ręczny. terraform destroy — usuwa całą infrastrukturę.

6.4 add-service.yml — Add Service

Wejścia: nazwa, vpn_ip, path, local_port, has_domain, domain, public_port, basic_auth

Kroki: walidacja (nazwa, IP w 10.8.0.0/24, DNS, unikalność portu/domeny/backendu) → zapis services/{name}.yml → terraform-sync-nsg.sh → ansible/add-service.yml

6.5 update-service.yml — Update Service

Podobnie do add, ale nadpisuje istniejący plik. Wykrywa zmianę domeny i usuwa stary certyfikat certbot. Na końcu: sync NSG + ansible/update-service.yml.

6.6 delete-service.yml — Delete Service

Usuwa certyfikat (jeśli domena), kasuje plik YAML, sync NSG, ansible/update-service.yml (przebudowa nginx/compose bez usługi).

6.7 list-services.yml — List Services

SSH na VM, Python wczytuje wszystkie services/*.yml i wypisuje tabelę usług.

6.8 update_domains.yml — Update Domains

Zmiana domen WG/Kuma. Puste wg_domain → FQDN Azure. Sync NSG (tryb single/dual), usuwanie starych certów, aktualizacja .bootstrap.env, ansible/update-domains.yml.

6.9 update-proxy-credentials.yml — Update Auth Password

Regeneruje nginx/.htpasswd z sekretów NGINX_BASIC_AUTH_USER / NGINX_BASIC_AUTH_PASSWORD.

7. Ansible — playbooki i role

7.1 Playbooki

Playbook	Role (kolejność)	Kiedy używany
bootstrap.yml	load_config → common → docker → directories → load_services → domain_validation → deploy → certbot → containers → routing → security	Pierwsze uruchomienie VM
deploy.yml	load_config → docker_access → directories → load_services → deploy → certbot → containers → routing	Push na main
add-service.yml	load_config → docker_access → load_services → domain_validation → deploy → certbot → containers	Nowa usługa
update-service.yml	jak add-service	Edycja / usunięcie usługi
update-domains.yml	jak add-service	Zmiana domen systemowych
maintenance.yml	common_update	Konserwacja apt (brak workflow — ręcznie)

7.2 Format pliku usługi (services/*.yml)

```
name: moja-aplikacja
vpn_ip: 10.8.0.2
path: /
local_port: 8080
has_domain: false
domain: ""
public_port: 9090
basic_auth: true
```

7.3 Role Ansible — szczegóły

load_config

Czyta `/opt/{REPO}/.bootstrap.env`, parsuje do słownika `bootstrap_config`. Wszystkie inne role z tego korzystają.

common (tylko bootstrap)

`apt update/upgrade`, instalacja `git`, `curl`, `python3`, `pip`, `apache2-utils`.

common_update (maintenance)

Aktualizacja pakietów, `autoremove`, `autoclean`.

docker (tylko bootstrap)

Repozytorium Docker CE, instalacja `docker-ce` i `compose` plugin, włączenie usługi `docker`.

docker_access

Dodaje użytkownika `azureuser` do grupy `docker`.

directories

Tworzy katalogi: `data/wireguard`, `data/kuma`, `certbot/conf`, `certbot/www`, `nginx`, `services`. Ustawia właściciela `services/` na `azureuser`.

load_services

Wczytuje wszystkie `services/*.yaml` do faktów: `vpn_services`, `service_domains`, `domain_services`.

domain_validation

Sprawdza przez `dig`, czy domeny WG (i Kuma w trybie dual) oraz domeny usług wskazują na publiczny IP serwera.

deploy

Renderuje szablon `docker-compose-template.yaml.j2` → `docker-compose.yaml`. Definiuje kontenery `wg-easy`, `uptime-kuma`, `nginx` (porty `80/443/8443/dynamiczne` `public_port`), `certbot` oraz sieć `bridge` `DOCKER_SUBNET`.

certbot

Trzy pliki zadań:

- **system-certificates.yml** — certyfikaty dla WG i Kuma
- **service-certificates.yml** — certyfikaty per usługa z domeną
- **renewal.yml** — cron codziennie o 03:00 na `certbot renew`

Szablony nginx:

- `nginx-bootstrap.conf.j2` — tymczasowy config pod ACME (odpowiedź „bootstrap”)
- `nginx-single-domain.conf.j2` — WG na 443, Kuma na 8443, proxy usług
- `nginx-dual-domain.conf.j2` — osobne domeny WG i Kuma; obsługa `public_port` bez domeny

containers

`docker compose up -d` w katalogu repo na VM.

routing

Instaluje usługę `systemd wg-route.service` — trasa `10.8.0.0/24` przez IP kontenera WG-Easy. Czeka do 120 s na dostępność gateway. Uruchamiana **po** starcie kontenerów.

security (tylko bootstrap)

Zapisuje `.env` z `INIT_ENABLED=false` — wyłącza ponowną inicjalizację WG-Easy po pierwszym bootstrapie.

8. Pliki tworzone na maszynie wirtualnej

Te pliki nie są w repozytorium Git — powstają przy bootstrapie i operacjach:

Ścieżka na VM	Źródło	Rola
/opt/{REPO}/.bootstrap.env	bootstrap-vm.yml	Stała konfiguracja: domeny, tryb, podsieci, IP kontenerów
/opt/{REPO}/.env	.env.template + envsubst	Runtime WG-Easy
/opt/{REPO}/docker-compose.yml	Ansible deploy	Definicja kontenerów
/opt/{REPO}/nginx/nginx.conf	Ansible certbot	Konfiguracja reverse proxy
/opt/{REPO}/nginx/.htpasswd	bootstrap / update-proxy-credentials	Basic auth nginx
/opt/{REPO}/services/*.yml	workflowy add/update/delete	Definicje usług użytkownika
/opt/{REPO}/ansible/inventory.ini	bootstrap-vm.yml	Inventory Ansible (localhost)
/opt/.github-bootstrap-completed	bootstrap-vm.yml	Zabezpieczenie przed ponownym bootstrapem

9. Tryby wdrożenia (single / dual)

Aspekt	Single	Dual
Warunek	Brak kuma_domain	Podany kuma_domain
Domena WG	wg_domain lub FQDN Azure	wg_domain lub FQDN Azure
Domena Kuma	Ta sama co WG	Osobna domena
Port Kuma	8443 (osobny listener nginx)	443 (osobny server block)
NSG	Otwarty port 8443	Bez 8443

Certyfikaty Jeden cert na WG (+ Kuma przez ten sam) Osobne certy WG i Kuma

10. Sekrety i zmienne GitHub

Variables (publiczne w repo settings)

Nazwa	Źródło
AZURE_CLIENT_ID	bootstrap_terraform.py → App Registration
AZURE_TENANT_ID	az account show
AZURE_SUBSCRIPTION_ID	az account show
SSH_PUBLIC_KEY	output/id_ed25519.pub

Secrets

Nazwa	Opis
SSH_PRIVATE_KEY	Klucz prywatny SSH (cała zawartość pliku)
WG_ADMIN_USER / WG_ADMIN_PASSWORD	Login do panelu WG-Easy
LETSENCRYPT_EMAIL	Email do Let's Encrypt
NGINX_BASIC_AUTH_USER NGINX_BASIC_AUTH_PASSWORD	/ Basic auth dla proxy

11. Mapa zależności — co z czego wynika

Źródło	Cel	Mechanizm
bootstrap_terraform.py	terraform/backend.tf	Zapis pliku

bootstrap_terraform.py	GitHub OIDC	Azure AD federated credential
bootstrap-vm.yml	VM + .bootstrap.env	Terraform + SSH
services/*.yml	docker-compose + nginx	Ansible load_services → deploy + certbot
services/*.yml	Reguły NSG	collect-public-ports → terraform-sync-nsg
.bootstrap.env	Wszystkie role Ansible	load_config
deploy.yml (workflow)	ansible/deploy.yml	git pull + SSH
wg-route.service	Komunikacja z klientami VPN	Trasa hosta → kontener WG-Easy

Przebieg dodania usługi bez domeny (public_port)

1. Użytkownik uruchamia **Add Service** z `has_domain=false`, `public_port=9090`
2. Workflow zapisuje `services/apka.yml` na VM
3. `terraform-sync-nsg.sh` dodaje port 9090 do NSG w Azure
4. Ansible regeneruje `docker-compose.yml` (mapowanie 9090:9090) i `nginx.conf` (listen 9090 → proxy na `vpn_ip:local_port`)
5. `docker compose up -d` stosuje zmiany

Koniec sprawozdania. Plik wygenerowany automatycznie z repozytorium **validationtest**.